

Every Equation in Your Thesis, Explained

Plain-language walkthrough of all numbered equations. Each card shows the equation with proper math symbols, what every symbol means, and a one-sentence explanation.

Chapter 1 — Signal Model

EQ 1.1 Additive Noise Model

$$y(t) = x(t) + n(t)$$

$y(t)$	The noisy signal you record with a microphone
$x(t)$	The clean speech you want to recover
$n(t)$	The background noise (traffic, fans, chatter, etc.)
t	Time (a continuous variable)

In plain words: The noisy recording is simply clean speech plus noise added on top. The entire goal of speech enhancement is to undo this addition — estimate $x(t)$ given only $y(t)$.

EQ 1.2 STFT Domain Additive Model

$$Y(t, f) = X(t, f) + N(t, f)$$

$Y(t, f)$	Noisy signal in the time-frequency domain (spectrogram)
$X(t, f)$	Clean speech spectrogram
$N(t, f)$	Noise spectrogram
t	Time frame index (e.g., frame 0, 1, 2, ... 62)
f	Frequency bin index (e.g., bin 0, 1, 2, ... 256)

In plain words: Same idea as Eq 1.1 but after the STFT. Instead of one signal over time, we now have a 2D grid (spectrogram) where each cell is a complex number representing energy at a specific time and frequency. The addition still holds at every cell.

EQ 1.3 Power Spectrum Additivity

$$|Y(t, f)|^2 \approx |X(t, f)|^2 + |N(t, f)|^2$$

$ \cdot ^2$	Power (squared magnitude) of a complex number
-------------	---

In plain words: If speech and noise are uncorrelated (independent), the power of the noisy signal roughly equals speech power plus noise power. This approximation is what makes spectral subtraction possible.

Chapter 1 — Classical Enhancement Methods

EQ 1.4 Basic Spectral Subtraction

$$|\hat{X}(t, f)|^2 = |Y(t, f)|^2 - |\hat{N}(f)|^2$$

$ \hat{X} ^2$	Estimated clean speech power
$ Y ^2$	Measured noisy power
$ \hat{N} ^2$	Estimated noise power (measured during silent segments)

In plain words: The simplest enhancement idea: subtract the estimated noise power from the noisy power. What remains should be the clean speech. The noise estimate is typically measured during pauses when no one is speaking.

EQ 1.5 Spectral Subtraction with Floor

$$|\hat{X}(t, f)|^2 = \max\left(|Y(t, f)|^2 - \alpha|\hat{N}(f)|^2, \beta|\hat{N}(f)|^2\right)$$

α	Over-subtraction factor (subtract more noise to be safe, typically > 1)
β	Spectral floor (never go below this – prevents negative power and “musical noise”)

In plain words: An improved version that prevents two problems: (1) subtracting too little noise (α makes it aggressive), and (2) getting negative or zero power, which creates annoying tonal artifacts called “musical noise” (β sets a minimum floor).

EQ 1.6 Phase Reconstruction

$$\hat{X}(t, f) = |\hat{X}(t, f)| \cdot e^{j\angle Y(t, f)}$$

$ \hat{X} $	Estimated clean magnitude (from subtraction)
$e^{j\angle Y}$	Phase of the noisy signal, reused as-is
j	Imaginary unit ($\sqrt{-1}$)
$\angle Y$	Phase angle of the noisy STFT coefficient

In plain words: After estimating the clean magnitude, we still need a phase to reconstruct the time-domain signal. Classical methods simply reuse the noisy signal's phase, assuming noise doesn't distort phase much. This is a key limitation – phase matters, especially at low SNR.

EQ 1.7 Wiener Filter Application

$$\hat{X}(t, f) = H_W(t, f) \cdot Y(t, f)$$

$H_W(t, f)$	Wiener filter gain — a number between 0 and 1
$Y(t, f)$	Noisy signal (complex-valued)
$\hat{X}(t, f)$	Enhanced signal

In plain words: Instead of subtracting noise, multiply each time-frequency point by a gain. Gain near 1 = mostly speech (keep it). Gain near 0 = mostly noise (suppress it). This is the masking idea that the entire thesis builds on.

EQ 1.8 Wiener Filter Gain Formula

$$H_W(t, f) = \frac{P_X(t, f)}{P_X(t, f) + P_N(t, f)}$$

P_X	Clean speech power spectrum
P_N	Noise power spectrum

In plain words: The gain is the fraction of total power that comes from speech. If a bin is 90% speech power → gain = 0.9 (keep most of it). If a bin is 10% speech power → gain = 0.1 (suppress most of it). This is the SNR-based soft mask.

EQ 1.9 Wiener Filter as Optimal Estimator

$$H_W(t, f) = \arg \min_H \mathbb{E}[|X(t, f) - H \cdot Y(t, f)|^2]$$

$\arg \min_H$	Find the value of H that minimizes...
$\mathbb{E}[\cdot]$	Expected value (average over all possible signals)
$ X - HY ^2$	Squared error between the true clean signal and the filtered output

In plain words: The Wiener filter is mathematically optimal — it is the single gain value that minimizes the average squared error between the true clean signal and the filtered output. Eq 1.8 is the solution to this optimization.

EQ 1.10

Wiener Filter via A Priori SNR

$$H_W(t, f) = \frac{\xi(t, f)}{1 + \xi(t, f)}$$

 $\xi(t, f)$ A priori SNR = P_X/P_N – the signal-to-noise ratio at this time-frequency bin

In plain words: Same Wiener filter, rewritten using the SNR. When SNR is high ($\xi \gg 1$), gain $\rightarrow 1$ (pass through). When SNR is low ($\xi \ll 1$), gain $\rightarrow 0$ (suppress). The a priori SNR is the core quantity that all classical methods estimate.

EQ 1.11

Decision-Directed SNR Estimation

$$\hat{\xi}(t, f) = \alpha_{\text{dd}} \frac{|\hat{X}(t-1, f)|^2}{P_N(t, f)} + (1 - \alpha_{\text{dd}}) \max(\hat{\gamma}(t, f) - 1, 0)$$

 $\hat{\xi}$

Estimated a priori SNR

 α_{dd}

Smoothing constant (typically 0.95–0.99) – how much to trust the previous frame

 $|\hat{X}(t-1)|^2$

Previous frame's enhanced speech power

 $\hat{\gamma}$ A posteriori SNR = $|Y|^2/P_N$ (instantaneous noisy SNR)

In plain words: You can't directly observe the true SNR, so estimate it by blending two sources: (1) last frame's enhancement result (stable but lagged), and (2) the current frame's raw SNR (noisy but current). The smoothing constant α controls the tradeoff.

EQ 1.12

MMSE Spectral Amplitude Estimator

$$\hat{A}(t, f) = \mathbb{E}[|X(t, f)| \mid Y(t, f)]$$

 $\hat{A}$

Estimated clean speech magnitude

 $\mathbb{E}[|X| \mid Y]$

Expected (average) magnitude of clean speech, given the noisy observation

In plain words: Instead of estimating power (squared magnitude), estimate the magnitude directly. This conditional expectation is the optimal estimate under the mean-squared-error criterion. The formula is the goal; Eq 1.13 is the actual solution.

EQ 1.13 Ephraim–Malah MMSE-STSA Solution

$$\hat{A} = \Gamma(1.5) \frac{\sqrt{v}}{\hat{\gamma}} \exp\left(-\frac{v}{2}\right) \left[(1+v) I_0\left(\frac{v}{2}\right) + v I_1\left(\frac{v}{2}\right) \right] |Y|$$

$\Gamma(1.5)$	Gamma function at 1.5 ≈ 0.886 (a constant)
v	$v = \xi\gamma/(1 + \xi)$ – combines a priori and a posteriori SNR
I_0, I_1	Modified Bessel functions of the first kind (special math functions)

In plain words: The closed-form solution to Eq 1.12 under the assumption that both speech and noise STFT coefficients follow Gaussian distributions. Don't memorize this formula – just know it computes the optimal spectral magnitude gain using SNR estimates. The Bessel functions arise from the circular symmetry of complex Gaussian variables.

EQ 1.14 MMSE Log-Spectral Amplitude Gain

$$G_{\text{LSA}} = \frac{\xi}{1 + \xi} \exp\left(\frac{1}{2} \int_v^\infty \frac{e^{-u}}{u} du\right)$$

G_{LSA}	The gain applied to the noisy magnitude
$\int_v^\infty$	Exponential integral – a well-known special function

In plain words: A perceptually better variant that minimizes error in the log-magnitude domain. The logarithmic scale matches how humans perceive loudness (we hear ratios, not differences). This produces less “musical noise” than Eq 1.13.

EQ 1.15A Subspace Method

$$\hat{\mathbf{x}} = \mathbf{H}\mathbf{y}$$

$\mathbf{y}$	Noisy signal vector (a chunk of samples)
$\mathbf{H}$	Filter matrix designed using eigenvalue decomposition
$\hat{\mathbf{x}}$	Estimated clean signal

In plain words: Subspace methods treat signal and noise as living in different subspaces (directions) of a high-dimensional space. The matrix $\mathbf{H}$ projects the noisy signal onto the speech subspace and suppresses the noise subspace.

EQ 1.15B Non-negative Matrix Factorization

$$|\mathbf{Y}| \approx \mathbf{W}\mathbf{H} = \mathbf{W}_s\mathbf{H}_s + \mathbf{W}_n\mathbf{H}_n$$

$ \mathbf{Y} $	Magnitude spectrogram of noisy signal
$\mathbf{W}_s, \mathbf{W}_n$	Learned spectral templates (basis) for speech and noise
$\mathbf{H}_s, \mathbf{H}_n$	Activation weights (how much each template is used at each time)

In plain words: NMF decomposes the spectrogram into learned building blocks. Speech has its own spectral templates (vowel shapes, consonant bursts), noise has different ones. By separating the activations, you can reconstruct speech alone.

Chapter 1 — Neural Network Enhancement & Masks

EQ 1.15 General Masking Framework

$$\hat{X}(t, f) = M(t, f) \cdot Y(t, f)$$

$M(t, f)$	Time-frequency mask (any type: binary, ratio, complex)
-----------	--

In plain words: The universal framework for speech enhancement: multiply the noisy spectrogram by a mask. Everything from the classical Wiener filter to modern neural networks fits this pattern. The only question is how to estimate the mask M .

EQ 1.16 MSE Training Loss

$$\mathcal{L}_{\text{MSE}} = \frac{1}{TF} \sum_{t=1}^T \sum_{f=1}^F \left| \hat{X}(t, f) - X(t, f) \right|^2$$

$\mathcal{L}$	Loss function — what the neural network tries to minimize during training
T	Number of time frames (e.g., 63 for 1-second training segments)
F	Number of frequency bins (257)
$\hat{X}$	Network's predicted clean spectrogram
X	True clean spectrogram (known during training)

In plain words: The average squared difference between what the network predicts and the actual clean speech, summed over every point in the spectrogram. Smaller = better. The network adjusts its weights to make this number as small as possible.

EQ 1.31 Ideal Binary Mask (IBM)

$$M_{\text{IBM}}(t, f) = \begin{cases} 1 & \text{if } \text{SNR}(t, f) > \theta \\ 0 & \text{otherwise} \end{cases}$$

θ SNR threshold (e.g., 0 dB)

In plain words: The simplest mask: each time-frequency bin is either fully kept (1) or fully removed (0). If local SNR is above the threshold, the bin is speech-dominated – keep it. Otherwise, it’s noise – remove it. Simple but creates harsh artifacts at boundaries.

EQ 1.32 Ideal Ratio Mask (IRM)

$$M_{\text{IRM}}(t, f) = \left(\frac{|X(t, f)|^2}{|X(t, f)|^2 + |N(t, f)|^2} \right)^\beta$$

β Tuning exponent – typically 0.5 (square root) for optimal results

In plain words: A soft version of IBM. Instead of 0 or 1, the mask is a continuous value between 0 and 1, proportional to how much speech energy is in each bin. This is essentially the Wiener filter gain – the neural network learns to predict this from noisy input.

EQ 1.33 Complex Ratio Mask

$$M_c(t, f) = M_r + jM_i$$

M_r Real part of the complex mask

M_i Imaginary part of the complex mask

j Imaginary unit

In plain words: A complex-valued mask that modifies both magnitude AND phase. When multiplied by the noisy signal, cross-terms rotate the phase: $\hat{X}_r = M_r Y_r - M_i Y_i$, $\hat{X}_i = M_r Y_i + M_i Y_r$. This is the key to phase correction.

EQ 1.36 Optimal Complex IRM

$$M_{\text{cIRM}} = \frac{Y_r X_r + Y_i X_i}{|Y|^2} + j \frac{Y_r X_i - Y_i X_r}{|Y|^2}$$

Y_r, Y_i	Real and imaginary parts of noisy STFT
X_r, X_i	Real and imaginary parts of clean STFT

In plain words: The mathematically optimal complex mask – if you could compute this exactly, you'd perfectly recover the clean speech (both magnitude and phase). It's derived by solving $M \cdot Y = X$ for M . In practice, a neural network tries to estimate this.

EQ 1.36A Bounded cIRM (tanh compression)

$$\hat{M}_r = K \cdot \tanh(m_r), \quad \hat{M}_i = K \cdot \tanh(m_i)$$

m_r, m_i	Raw (unbounded) network outputs
$\tanh$	Hyperbolic tangent – squishes any value into (-1, +1)
K	Compression limit (typically 10), so the mask stays in [-K, K]

In plain words: The raw network output can be any number, but the mask needs bounding for stable training. The tanh function smoothly compresses the output. $K=10$ allows mask values up to ± 10 , necessary because the optimal cIRM can exceed 1.

EQ 1.37 Phase-Sensitive Mask

$$M_{\text{PSM}} = \frac{|X|}{|Y|} \cos(\theta_x - \theta_y)$$

$ X / Y $	Ratio of clean to noisy magnitudes
$\cos(\theta_x - \theta_y)$	Cosine of the phase difference between clean and noisy signals

In plain words: A middle ground: still a real-valued mask (applied to magnitude only), but it accounts for phase differences through the cosine factor. When clean and noisy phases align ($\cos \approx 1$), it behaves like IRM. When they differ, it adjusts.

Chapter 1 — Transformer Architecture

EQ 1.18 Query, Key, Value Projections

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}^V$$

$\mathbf{Z}$	Input sequence — a matrix where each row is one frame's features ($T \times d$)
$\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$	Learned weight matrices that project input into three different “views”
$\mathbf{Q}$	Queries — “what am I looking for?”
$\mathbf{K}$	Keys — “what do I contain?”
$\mathbf{V}$	Values — “what information do I provide?”

In plain words: Each frame is linearly projected into three representations. Think of a library: Q is your search query, K is the index card of each book, V is the book's content. You match your query against all index cards, then read the relevant books.

EQ 1.19 Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

$\mathbf{Q}\mathbf{K}^\top$	Dot product of every query with every key → similarity scores ($T \times T$ matrix)
$\sqrt{d_k}$	Scaling factor ($d_k = 32$) — prevents dot products from getting too large
softmax	Converts scores to probabilities (all positive, sum to 1 per row)

In plain words: The core of the Transformer. For each frame, compute how similar its query is to every other frame's key, normalize to get attention weights, then take a weighted average of all values. This lets every frame “attend to” every other frame in the sequence.

EQ 1.20 Multi-Head Attention

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

h	Number of heads (8 in this thesis)
head_i	Output of attention head i
Concat	Stack all heads side by side
$\mathbf{W}^O$	Output projection that combines all heads

In plain words: Instead of one big attention, run 8 smaller ones in parallel (each with $d/8 = 32$ dimensions). Each head can learn to attend to different things — nearby frames, harmonic structure, noise patterns. Concatenate results and project back.

EQ 1.22 **Feed-Forward Network**

$$\text{FFN}(\mathbf{z}) = \mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$$

$\mathbf{W}_1$	First weight matrix — expands from 256 to 1024 dimensions
ReLU	Activation function: $\max(0, x)$ — introduces nonlinearity
$\mathbf{W}_2$	Second weight matrix — compresses from 1024 back to 256
$\mathbf{b}_1, \mathbf{b}_2$	Bias vectors

In plain words: After attention mixes information across frames, the FFN processes each frame independently. It expands to 4× dimension (256 → 1024), applies a nonlinearity, then compresses back (1024 → 256). This adds the nonlinear processing power that attention alone lacks.

EQ 1.23 **Residual Connection + LayerNorm**

$$\mathbf{z}' = \text{LayerNorm}(\mathbf{z} + \text{Sublayer}(\mathbf{z}))$$

$\mathbf{z}$	Input to the sublayer
$\text{Sublayer}(\mathbf{z})$	Either multi-head attention or FFN
$\mathbf{z} + \text{Sublayer}(\mathbf{z})$	Residual connection — add input back to the output
LayerNorm	Normalize each vector to zero mean and unit variance

In plain words: Two stabilization tricks: (1) Residual connection (the + sign) means the network only needs to learn what to ADD, not rebuild from scratch. (2) Layer normalization prevents values from growing or shrinking too much across layers. Together these make deep Transformers trainable.

Chapter 1 — Positional Encoding

EQ 1.24

Sinusoidal Positional Encoding

$$\text{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d}}\right), \quad \text{PE}(p, 2i+1) = \cos\left(\frac{p}{10000^{2i/d}}\right)$$

p	Position index (0, 1, 2, ... = which frame in the sequence)
i	Dimension index within the encoding vector
d	Model dimension (256 in this thesis)
10000	Base frequency — controls how fast the waves oscillate

In plain words: Without positional encoding, the Transformer sees frames as a bag, not a sequence. This creates a unique “fingerprint” for each position using sine and cosine waves at different frequencies. Low dimensions oscillate fast (local patterns), high dimensions oscillate slowly (global position). This is the ABSOLUTE encoding — it assigns fixed patterns to fixed positions, which is the root cause of the length generalization problem.

EQ 1.25

Adding PE to Input

$$\mathbf{z}_p = \mathbf{e}_p + \text{PE}(p)$$

$\mathbf{e}_p$	Content embedding at position p (from input projection)
$\text{PE}(p)$	Positional encoding vector for position p

In plain words: Add the positional encoding to the input embedding. After this, the representation contains both WHAT the frame is (spectral content) and WHERE it is (position). The problem: if the model only sees positions 0–62 during training, positions 63+ at test time are unknown territory.

EQ 1.27

Shaw et al. Relative Attention

$$e_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j + \mathbf{q}_i^\top \mathbf{a}_{ij}^K + b_{ij}}{\sqrt{d_k}}$$

$\mathbf{q}_i^\top \mathbf{k}_j$	Content-content similarity (same as standard attention)
$\mathbf{q}_i^\top \mathbf{a}_{ij}^K$	Content-position interaction — how the query relates to the relative distance
$\mathbf{a}_{ij}^K$	Learned embedding for relative distance ($i - j$)

In plain words: Shaw’s method adds relative position information directly into the attention score. Instead of “what is at position 5?” it asks “what is 3 positions to my left?”. The embeddings are learned for each relative distance, but clipped beyond a maximum.

EQ 1.28A

Transformer-XL Decomposition

$$e_{ij} = \underbrace{\mathbf{q}_i^\top \mathbf{W}_E^\top \mathbf{W}_E \mathbf{k}_j}_{\text{content-content}} + \underbrace{\mathbf{q}_i^\top \mathbf{W}_E^\top \mathbf{W}_R \mathbf{r}_{i-j}}_{\text{content-position}} + \underbrace{\mathbf{u}^\top \mathbf{W}_E \mathbf{k}_j}_{\text{global content}} + \underbrace{\mathbf{v}^\top \mathbf{W}_R \mathbf{r}_{i-j}}_{\text{global position}}$$

Term (a)	How similar are the contents of positions i and j ?
Term (b)	How does position i 's content relate to its distance from j ?
Term (c)	A global bias toward certain content (same for all queries)
Term (d)	A global bias toward certain distances (same for all queries)
$\mathbf{r}_{i-j}$	Sinusoidal encoding of the relative distance
$\mathbf{u}, \mathbf{v}$	Learned global bias vectors

In plain words: Transformer-XL decomposes attention into four terms, cleanly separating content-based attention (“what are the frames about?”) from position-based attention (“how far apart are they?”). Terms (b) and (d) depend only on relative distance ($i - j$), enabling generalization to new positions.

EQ 1.29

Rotary Position Embedding (RoPE)

$$\text{RoPE}(\mathbf{x}, p) = \begin{pmatrix} x_0 \cos p\theta_0 - x_1 \sin p\theta_0 \\ x_0 \sin p\theta_0 + x_1 \cos p\theta_0 \\ \vdots \end{pmatrix}$$

p	Position index
θ_i	Rotation angle for dimension pair i
x_0, x_1	Pairs of dimensions in the embedding vector

In plain words: RoPE rotates each pair of dimensions by an angle proportional to position. The key property: the dot product between two RoPE-encoded vectors depends only on the DIFFERENCE in positions — giving relative position encoding for free through the geometry of rotation.

EQ 1.30

ALiBi (Attention with Linear Biases)

$$a_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} - m \cdot |i - j|$$

m	Fixed slope (different for each attention head)
$ i - j $	Absolute distance between positions
$-m \cdot i - j $	Linear penalty — farther frames get lower attention scores

In plain words: The simplest relative method: subtract a penalty proportional to distance. Nearby frames get high scores, distant frames are penalized. No learned parameters. But the LINEAR growth is the problem: at 25× longer sequences, the penalty grows 25×, which is out of distribution.

Chapter 2 — Proposed Method

CRPE

Convolutional Relative Position Encoding

$$\mathbf{h}^{(\ell)} = \mathbf{h}^{(\ell)} + \text{PointwiseConv}\left(\text{DepthwiseConv}\left(\text{LN}(\mathbf{h}^{(\ell)})\right)\right)$$

LN	Layer normalization
DepthwiseConv	1D convolution processing each channel independently (kernel size 31)
PointwiseConv	1×1 convolution that mixes channels
$\mathbf{h}^{(\ell)}$	Hidden representation at layer ℓ

In plain words: Instead of encoding position with a formula, use a convolution. A kernel of size 31 lets each frame “see” 15 frames left and 15 right (~496 ms). The convolution naturally encodes RELATIVE position because the same weights apply everywhere — translation-equivariant.

LRAB

Logarithmic Relative Attention Bias

$$b_{ij}^{(h)} = \alpha_h \cdot \text{sign}(i - j) \cdot \log(1 + |i - j|)$$

α_h	Learned scalar per attention head (controls distance sensitivity)
$\text{sign}(i-j)$	+1 if j is ahead, -1 if behind, 0 if same position
$\log(1+ i-j)$	Logarithm of distance – grows SLOWLY as distance increases
$b_{ij}^{(h)}$	Bias added to the attention score for head h

In plain words: The key innovation for length generalization. Like ALiBi (Eq 1.30), it adds a distance-based bias. But instead of LINEAR growth (which explodes at 25×), it uses LOGARITHMIC growth. Going from distance 62 to distance 1562 only increases the bias by ~78%, not 2500%. This keeps attention in a familiar range even for much longer inputs.

SCCM

Sigmoid-Compressed Complex Mask

$$M_{\text{mag}} = K \cdot \sigma(m_r), \quad M_{\text{phase}} = \exp(j \cdot \tanh(m_i) \cdot \pi)$$

K	Maximum magnitude gain ($K = 2$)
$\sigma(\cdot)$	Sigmoid function – smoothly maps any value to (0, 1)
m_r, m_i	Raw network outputs (can be any real number)
M_{mag}	Magnitude mask – bounded to (0, K) = (0, 2)
$\tanh(m_i) \cdot \pi$	Phase rotation – bounded to $(-\pi, +\pi)$

In plain words: A novel complex mask. The magnitude part uses sigmoid (always positive, capped at 2) instead of tanh (which allows negative values). The phase part independently rotates the angle. $K=2$ allows the mask to amplify speech that was partially cancelled by noise, while staying bounded for training stability.

LOSS

Combined Training Loss

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{mag}} + \beta \cdot \mathcal{L}_{\text{SI-SNR}}$$

$\alpha = 1.0$	Weight for magnitude loss
$\beta = 0.5$	Weight for SI-SNR loss
$\mathcal{L}_{\text{mag}}$	Mean squared error on the magnitude spectrogram
$\mathcal{L}_{\text{SI-SNR}}$	Negative scale-invariant SNR (time-domain quality measure)

In plain words: The network optimizes two objectives: (1) make the magnitude spectrogram match the clean one (frequency domain), and (2) make the reconstructed waveform sound like the clean one (time domain). Combining both gives better results than either alone.

PROP. 1**LRAB Extrapolation Bound**

$$b_{\max}(T_{\text{test}}) - b_{\max}(T_{\text{train}}) = |\alpha_h| \cdot \log\left(\frac{T_{\text{test}}}{T_{\text{train}}}\right)$$

$b_{\max}(T)$	Maximum bias value for sequence length T
$T_{\text{train}} = 63$	Training sequence length (1 second $\rightarrow$ 63 STFT frames)
T_{test}	Test sequence length (e.g., 1563 for 25 seconds)

In plain words: The formal guarantee: the “overshoot” (how far beyond the training range the bias goes) grows only logarithmically. For 25× longer sequences, the overshoot is only $\log(25) \approx 3.22$ times α_h , compared to the training range of $[0, 4.14 \cdot \alpha_h]$. Compare with ALiBi where the overshoot is LINEAR (25×), which is catastrophic.